

Playwright Storage State Guide

Complete Step-by-Step Implementation for OrangeHRM Site

PART 1 Initial Setup Commands (PowerShell)

Initialize project, install dependencies, create directory structure, and update .gitignore:

```
npm init -y npm init playwright@latest npx playwright install chromium # Create authentication state
```

PART 2 Project Structure

```
playwright-storage-demo/ ├── node_modules/ ├── playwright/ |   ├── .auth/ |   └── user.json #  
    Saved session state (cookies & localStorage) ├── tests/ |   ├── auth.setup.js # One-time login  
    script |   └── orangehrm.spec.js # Complete test suite (Pass, Fail, Negative) ├── .gitignore #  
    Ignores node_modules/ and playwright/.auth/ ├── package.json └── playwright.config.js #  
    Playwright project dependencies config
```

PART 3 Authentication Setup Script

File: `tests/auth.setup.js`

```
const { test: setup, expect } = require('@playwright/test'); const authFile = 'playwright/.auth/u
```

PART 4 Playwright Configuration

File: `playwright.config.js`

```
const { defineConfig, devices } = require('@playwright/test'); module.exports = defineConfig({ te
```

PART 5 Full Test Suite Implementation (Page 1)

File: `tests/orangehrm.spec.js`

```
const { test, expect } = require('@playwright/test'); test.describe('OrangeHRM Test Suite - Storage', () => {
```

PART 5 Full Test Suite Implementation (Page 2)

File: `tests/orangehrm.spec.js` (Continued)

```
// --- INTENTIONALLY FAILING SCENARIOS --- test('Failed Case: Incorrect Dashboard Heading Assertion', async () => {
```

PART 6 Execution & Workflow Overview

Execute the entire test suite from terminal using:

```
npx playwright test
```

EXECUTION FLOW SUMMARY

- **Step 1 (Setup Phase):** Playwright automatically triggers `tests/auth.setup.js` prior to running tests because of dependencies: `['setup']` in `playwright.config.js`.
- **Step 2 (State Saving):** The setup test logs in with Admin / admin123 and exports active session cookies and tokens into `playwright/.auth/user.json`.
- **Step 3 (Positive Tests):** Positive tests launch directly using saved `storageState` without performing any UI login steps.
- **Step 4 (Negative Tests):** Negative tests override the project config via `test.use({ storageState: { cookies: [], origins: [] } })` to test unauthenticated login validation messages.
- **Step 5 (Failing Scenarios):** The intentional failing assertions trigger failure reports for reporting tools (like Allure or HTML reports).